

CLASSIFICATION

BY: KELOMPOK 2

Alvia Zuhra	(2208107010003)
Al-Mahfuzh Fadhlur Rohman	(2208107010016)
Nurul Uzratun Nashriyyah	(2208107010030)
Ganang Setyo Hadi	(2208107010052)
Azimah Al-Huda	(2208107010069)

PEMAHAMAN DATASET

DESKRIPSI DATASET:

Dataset "Student's Performance" merupakan kumpulan data sintetis yang menggambarkan berbagai faktor yang memengaruhi prestasi akademik siswa SMA. Dengan jumlah 2.392 entri, dataset ini dirancang untuk keperluan penelitian pendidikan dan mencerminkan pola-pola nyata dalam dunia pendidikan. Data ini bersumber dari Kaggle dan dikembangkan oleh Rabie El Kharoua pada tahun 2024.

VARIABEL YANG DIGUNAKAN:

Terdiri dari 15 variabel utama:

- Demografi: StudentID, Age, Gender, Ethnicity
- Latar Belakang Keluarga: ParentalEducation, ParentalSupport
- Kebiasaan Belajar: StudyTimeWeekly, Absences, Tutoring
- Aktivitas Ekstrakurikuler: Extracurricular, Sports, Music, Volunteering
- Kinerja Akademik: GPA, GradeClass (target klasifikasi)

PEMAHAMAN DATASET

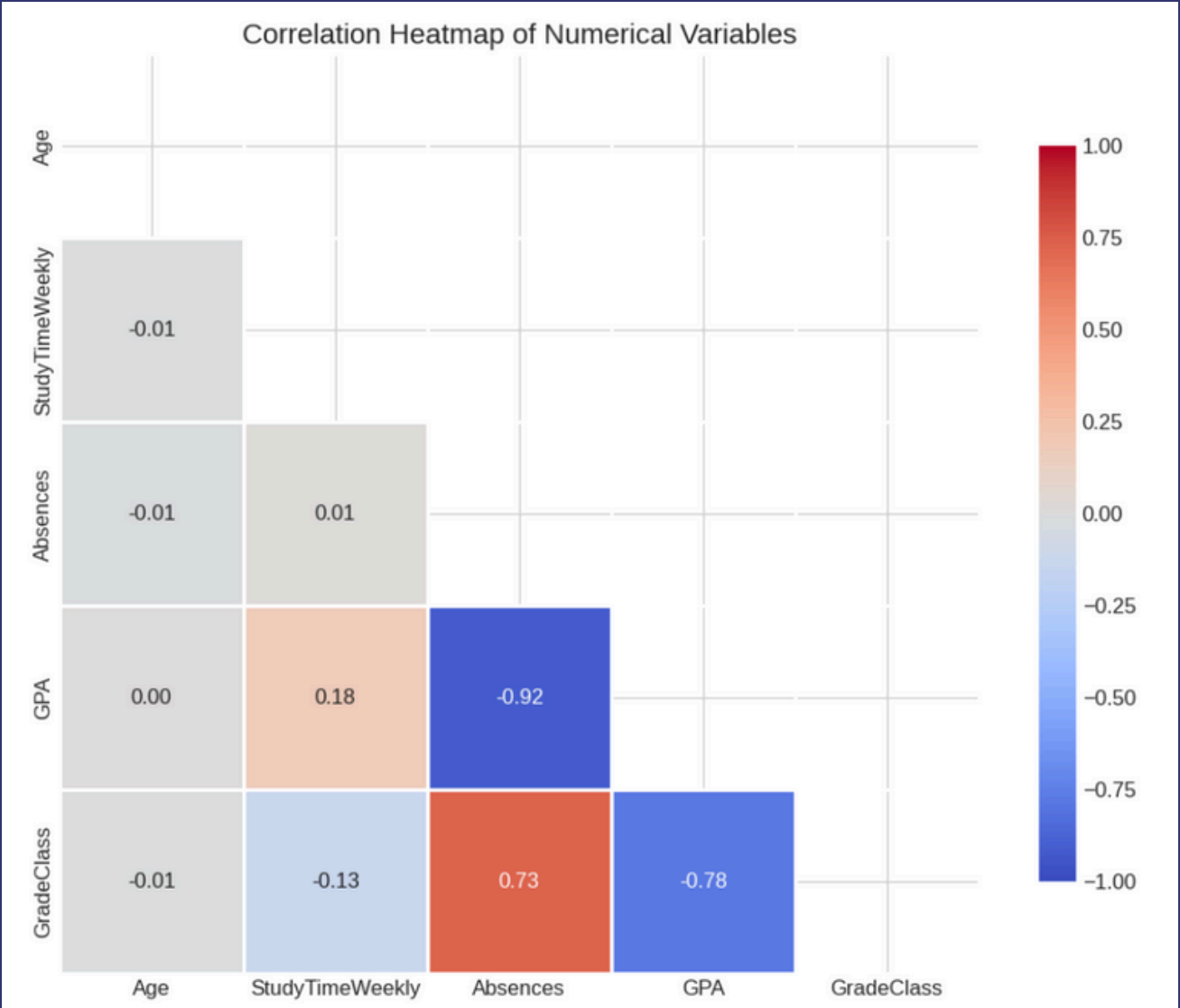
STATISTIK DESKRIPTIF DAN VISUALISASI AWAL



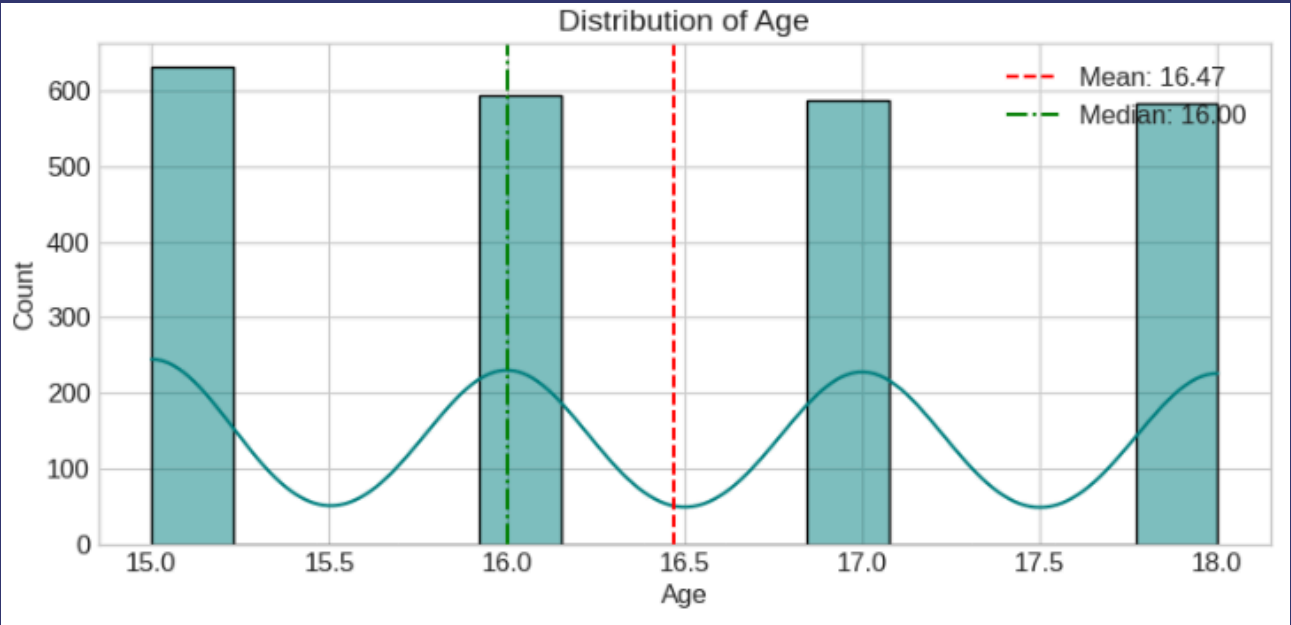
Statistik Deskriptif Variabel Numerik:

	count	mean	std	min	25%	50%	75%	max	range	variance	skewness	kurtosis
Age	2392.000	16.469	1.124	15.000	15.000	16.000	17.000	18.000	3.000	1.263	0.036	-1.372
StudyTimeWeekly	2392.000	9.772	5.653	0.001	5.043	9.705	14.408	19.978	19.977	31.954	0.050	-1.136
Absences	2392.000	14.541	8.467	0.000	7.000	15.000	22.000	29.000	29.000	71.697	-0.026	-1.176
GPA	2392.000	1.906	0.915	0.000	1.175	1.893	2.622	4.000	4.000	0.838	0.015	-0.863
GradeClass	2392.000	2.984	1.234	0.000	2.000	4.000	4.000	4.000	4.000	1.523	-0.901	-0.417

Kolerasi Antara Variabel Numerik:



Distribusi Variabel Numerik:



PEMAHAMAN DATASET

STATISTIK DESKRIPTIF DAN VISUALISASI AWAL



Distribusi Frekuensi Data Kategorikal:

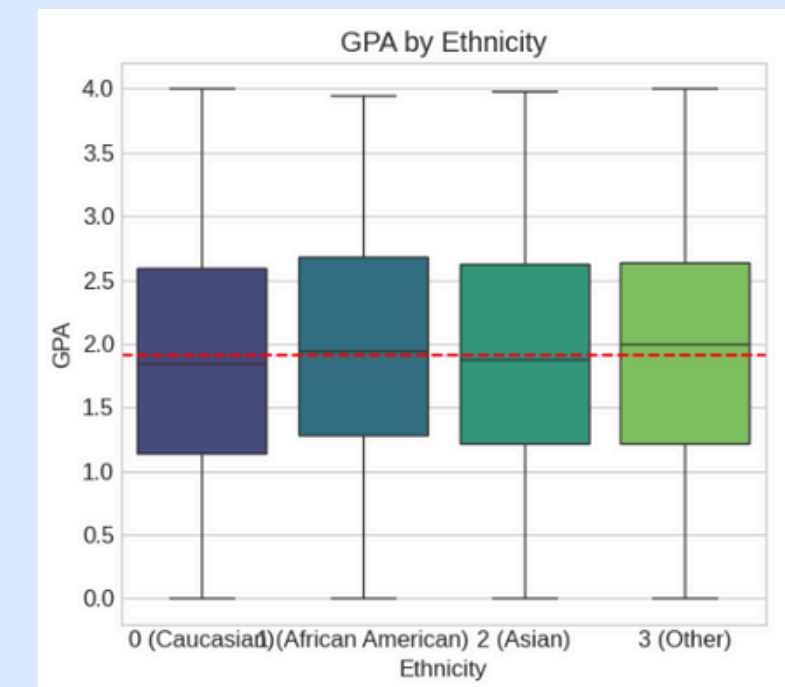
Ethnicity Distribution:

	Count	Percentage (%)
0 (Caucasian)	1207	50.460
1 (African American)	493	20.610
2 (Asian)	470	19.650
3 (Other)	222	9.280

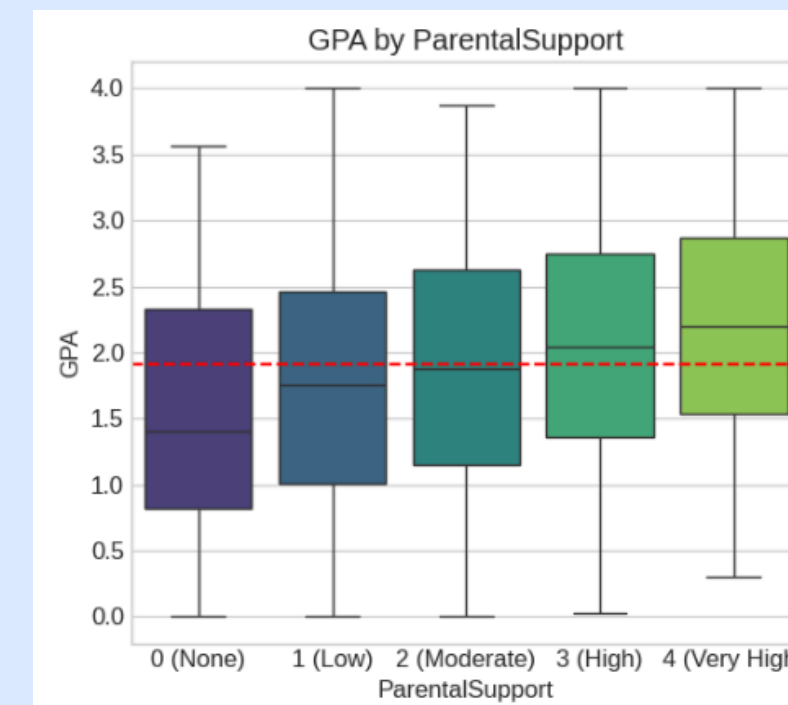
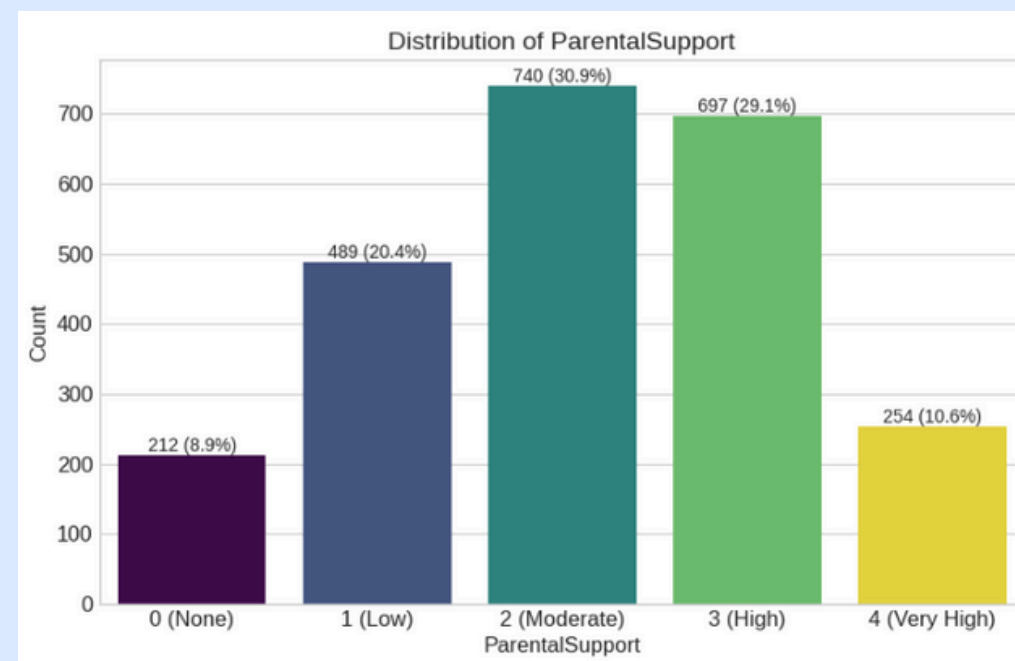
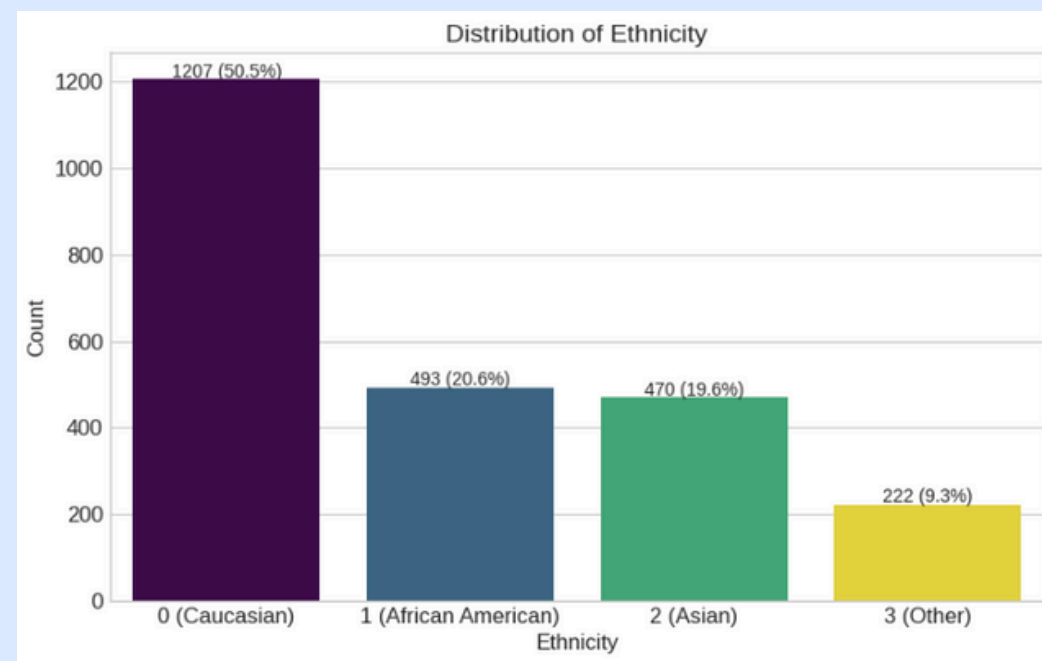
ParentalSupport Distribution:

	Count	Percentage (%)
0 (None)	212	8.860
1 (Low)	489	20.440
2 (Moderate)	740	30.940
3 (High)	697	29.140
4 (Very High)	254	10.620

Hubungan antara Variabel Kategorikal dan GPA:



Visualisasi Variabel Kategorikal:





EKSPLORASI DATA DAN PRAPEMROSESAN



1. Menghitung Fitur yang Tidak Relevan

	Age	Gender	Ethnicity	ParentalEducation	StudyTimeWeekly	Absences	Tutoring	ParentalSupport	Extracurricular	Sports	Music	Volunteering	GradeClass
0	17	1	0	2	19.834	7	1	2	0	0	1	0	2.000
1	18	0	0	1	15.409	0	0	1	0	0	0	0	1.000
2	15	0	2	3	4.211	26	0	2	0	0	0	0	4.000
3	17	1	0	3	10.029	14	0	3	1	0	0	0	3.000
4	17	1	0	2	4.672	17	1	3	0	0	0	0	4.000

Dengan menggunakan perintah `df.drop(['StudentID', 'GPA'], axis=1)`, kita menghapus kolom 'StudentID' dan 'GPA' dari DataFrame `df`.

2. Memeriksa Missing Values

```
Age          0
Gender       0
Ethnicity    0
ParentalEducation  0
StudyTimeWeekly  0
Absences     0
Tutoring     0
ParentalSupport  0
Extracurricular  0
Sports       0
Music        0
Volunteering  0
GradeClass   0
dtype: int64
```

Semua kolom bernilai 0, menandakan tidak ada data yang hilang pada DataFrame.

3. Ekspansi Polinomial

```
df['ParentalEducation_StudyTime'] = df['ParentalEducation'] * df['StudyTimeWeekly']
df['StudyAbsenceRatio'] = df['StudyTimeWeekly'] / (df['Absences'] + 1)

numerical_features = df[['Age', 'StudyTimeWeekly', 'Absences', 'ParentalEducation_StudyTime', 'StudyAbsenceRatio']]

poly = PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)
poly_features = poly.fit_transform(numerical_features)

poly_features_df = pd.DataFrame(poly_features, columns=poly.get_feature_names_out(numerical_features.columns))

df = pd.concat([df, poly_features_df], axis=1)
```

Ditambahkan dua kolom baru:

ParentalEducation_StudyTime dan StudyAbsenceRatio. Fitur numerik ditransformasi secara polinomial (derajat 2) dan digabung ke DataFrame untuk memperkaya analisis.



EKSPLORASI DATA DAN PRAPEMROSESAN



4. Menghapus Kolom Duplikat

```
## Remove duplicate columns if any  
df = df.loc[:, ~df.columns.duplicated()]
```

Menggunakan `df.loc[:, ~df.columns.duplicated()]` untuk menghapus kolom dengan nama duplikat dan mempertahankan hanya satu versi dari setiap kolom.

5. Data Splitting

```
X = df.drop('GradeClass', axis=1)  
y = df['GradeClass']  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

Memisahkan fitur (X) dan target (y = GradeClass), lalu membagi data menjadi 80% pelatihan dan 20% pengujian menggunakan `train_test_split` dengan `random_state=42` untuk hasil yang konsisten.

6. Scaling

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Fitur dinormalisasi agar rata-rata = 0 dan standar deviasi = 1 menggunakan `fit_transform` (training) dan `transform` (testing) untuk memastikan konsistensi skala dan meningkatkan akurasi model.

IMPLEMENTASI MODEL

TAHAPAN YANG DILAKUKAN:

1. Model Selection 🔍

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Naive Bayes
- Decision Tree

2. Hyperparameter Tuning ⚙️

- Menggunakan Grid Search untuk performa optimal

3. Model Evaluation 📊

- Evaluasi dengan: Accuracy, Precision, Recall, F1 Score, Confusion Matrix, ROC Curve

4. Feature Importance Analysis 📈

- Menentukan fitur paling berpengaruh menggunakan Logistic Regression & Decision Tree

IMPLEMENTASI MODEL

LOGISTIC REGRESSION

```
# Logistic Regression with hyperparameter tuning
print("\n🚦 LOGISTIC REGRESSION")
print("="*80)
print("Performing hyperparameter tuning for Logistic Regression...")

# Define parameter grid
lr_param_grid = {
    .... 'C': [0.01, 0.1, 1, 10, 100],
    .... 'penalty': ['l1', 'l2'],
    .... 'solver': ['liblinear'],
    .... 'class_weight': [class_weights_dict, None]
}

# Create and train GridSearchCV
lr_grid = GridSearchCV(
    .... LogisticRegression(max_iter=1000),
    .... param_grid=lr_param_grid,
    .... cv=cv_strategy,
    .... scoring='f1_macro',
    .... n_jobs=-1,
    .... verbose=1
)

lr_grid.fit(X_train_scaled, y_train)

# Get best model
lr_best = lr_grid.best_estimator_
print(f"Best parameters: {lr_grid.best_params_}")
print(f"Best cross-validation score: {lr_grid.best_score_:.4f}")

# Evaluate the model
lr_metrics = evaluate_model(lr_best, X_train_scaled, X_test_scaled, y_train, y_test, "Logistic Regression")
all_metrics.append(lr_metrics)
```

IMPLEMENTASI MODEL

K-NEAREST NEIGHBORS

```
# K-Nearest Neighbors with hyperparameter tuning
print("\n🔍 K-NEAREST NEIGHBORS")
print("="*80)
print("Performing hyperparameter tuning for K-NN...")

# Define parameter grid
knn_param_grid = {
    ... 'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
    ... 'weights': ['uniform', 'distance'],
    ... 'p': [1, 2] # 1 for Manhattan, 2 for Euclidean
}

# Create and train GridSearchCV
knn_grid = GridSearchCV(
    ... KNeighborsClassifier(),
    ... param_grid=knn_param_grid,
    ... cv=cv_strategy,
    ... scoring='f1_macro',
    ... n_jobs=-1,
    ... verbose=1
)

knn_grid.fit(X_train_scaled, y_train)

# Get best model
knn_best = knn_grid.best_estimator_
print(f"Best parameters: {knn_grid.best_params_}")
print(f"Best cross-validation score: {knn_grid.best_score_:.4f}")

# Evaluate the model
knn_metrics = evaluate_model(knn_best, X_train_scaled, X_test_scaled, y_train, y_test, "K-Nearest Neighbors")
all_metrics.append(knn_metrics)
```

IMPLEMENTASI MODEL

NAIVE BAYES

```
# Naive Bayes
print("\n🧠 NAIVE BAYES")
print("="*80)
print("Training Gaussian Naive Bayes model...")

# For Naive Bayes, we'll use cross-validation to assess performance
nb = GaussianNB()

# Perform cross-validation
cv_scores = cross_val_score(nb, X_train_scaled, y_train, cv=cv_strategy, scoring='f1_macro')
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.4f}")
print(f"Standard deviation: {cv_scores.std():.4f}")

# Train the model on the full training set
nb.fit(X_train_scaled, y_train)

# Evaluate the model
nb_metrics = evaluate_model(nb, X_train_scaled, X_test_scaled, y_train, y_test, "Naive Bayes")
all_metrics.append(nb_metrics)
```

IMPLEMENTASI MODEL

DECISION TREE

```
# Decision Tree with hyperparameter tuning
print("\n 🌳 DECISION TREE")
print("="*80)
print("Performing hyperparameter tuning for Decision Tree...")

# Define parameter grid
dt_param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [class_weights_dict, None]
}

# Create and train GridSearchCV
dt_grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid=dt_param_grid,
    cv=cv_strategy,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

dt_grid.fit(X_train_scaled, y_train)

# Get best model
dt_best = dt_grid.best_estimator_
print(f"Best parameters: {dt_grid.best_params_}")
print(f"Best cross-validation score: {dt_grid.best_score_:.4f}")

# Evaluate the model
dt_metrics = evaluate_model(dt_best, X_train_scaled, X_test_scaled, y_train, y_test, "Decision Tree")
all_metrics.append(dt_metrics)

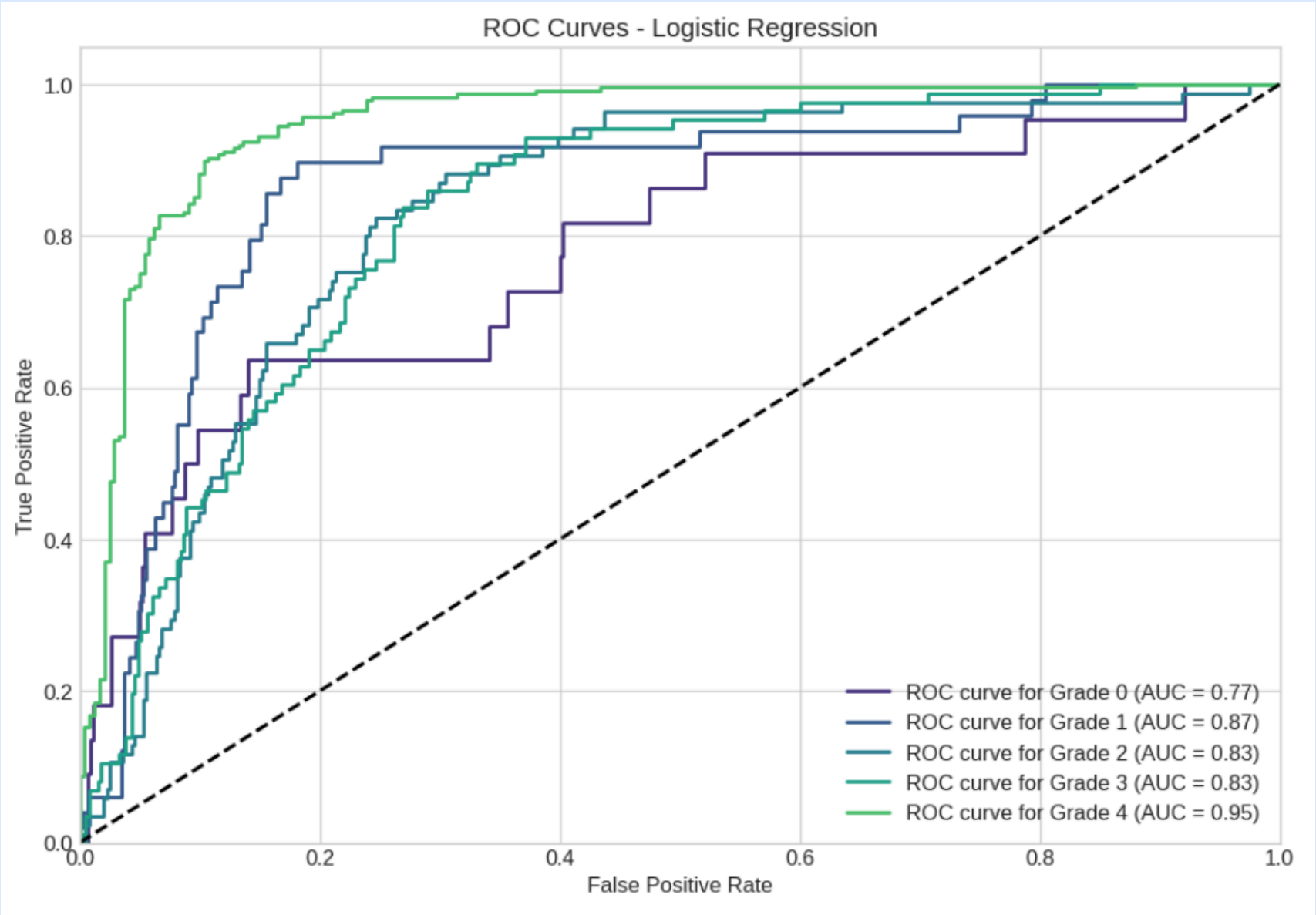
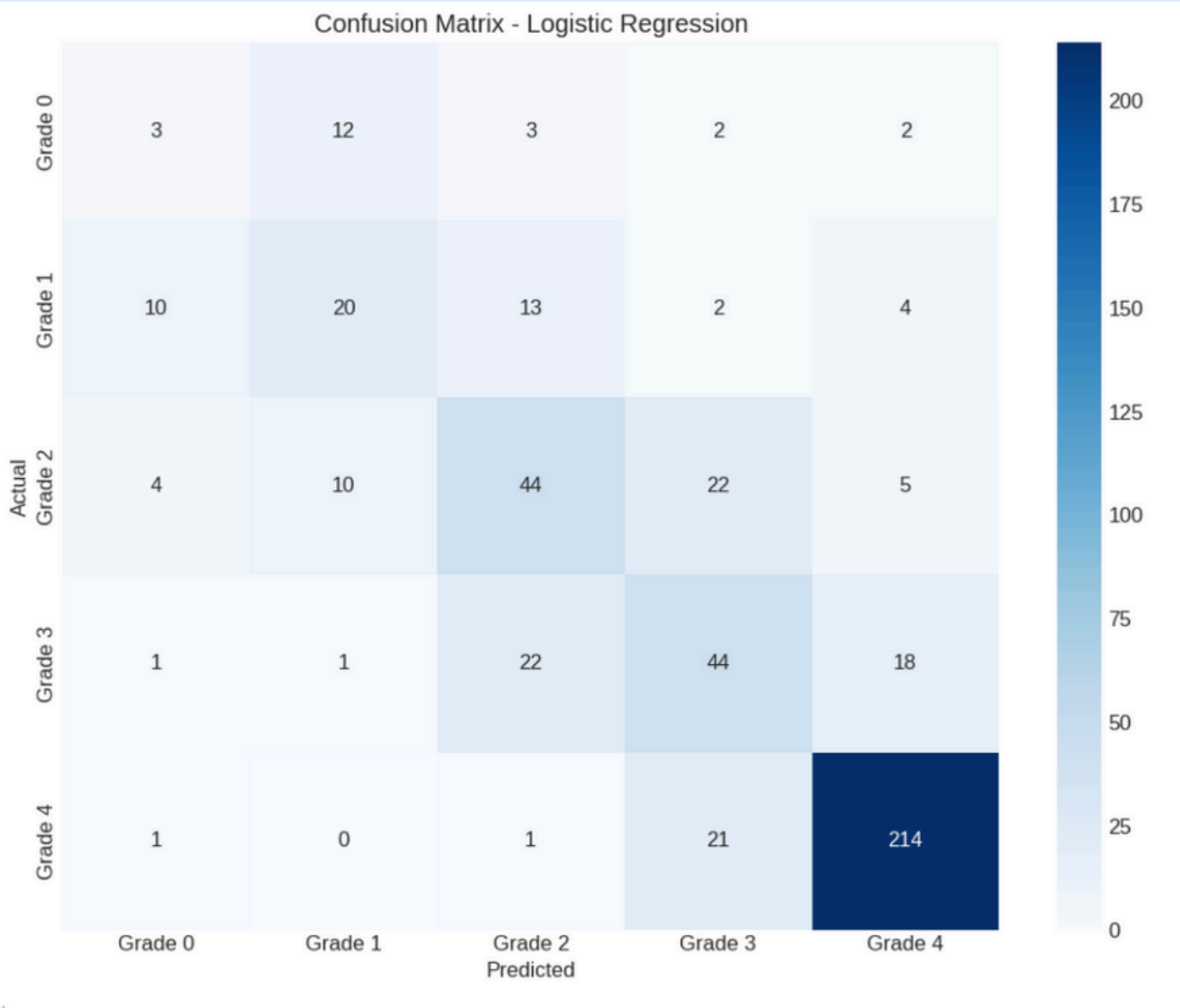
# Visualize the Decision Tree (limiting depth for readability)
plt.figure(figsize=(20, 12))
plot_tree(dt_best,
          max_depth=3,
          filled=True,
          feature_names=X_train.columns,
          class_names=[f'Grade-{i}' for i in range(len(np.unique(y_test)))],
          rounded=True)
plt.title("Decision Tree Visualization (Limited to Depth 3)")
plt.tight_layout()
plt.show()
```

EVALUASI MODEL

LOGISTIC REGRESSION DENGAN HYPERPARAMETER TUNING:

----- Logistic Regression Evaluation -----

Accuracy: 0.6785
Precision (macro): 0.5035
Recall (macro): 0.4954
F1 Score (macro): 0.4988

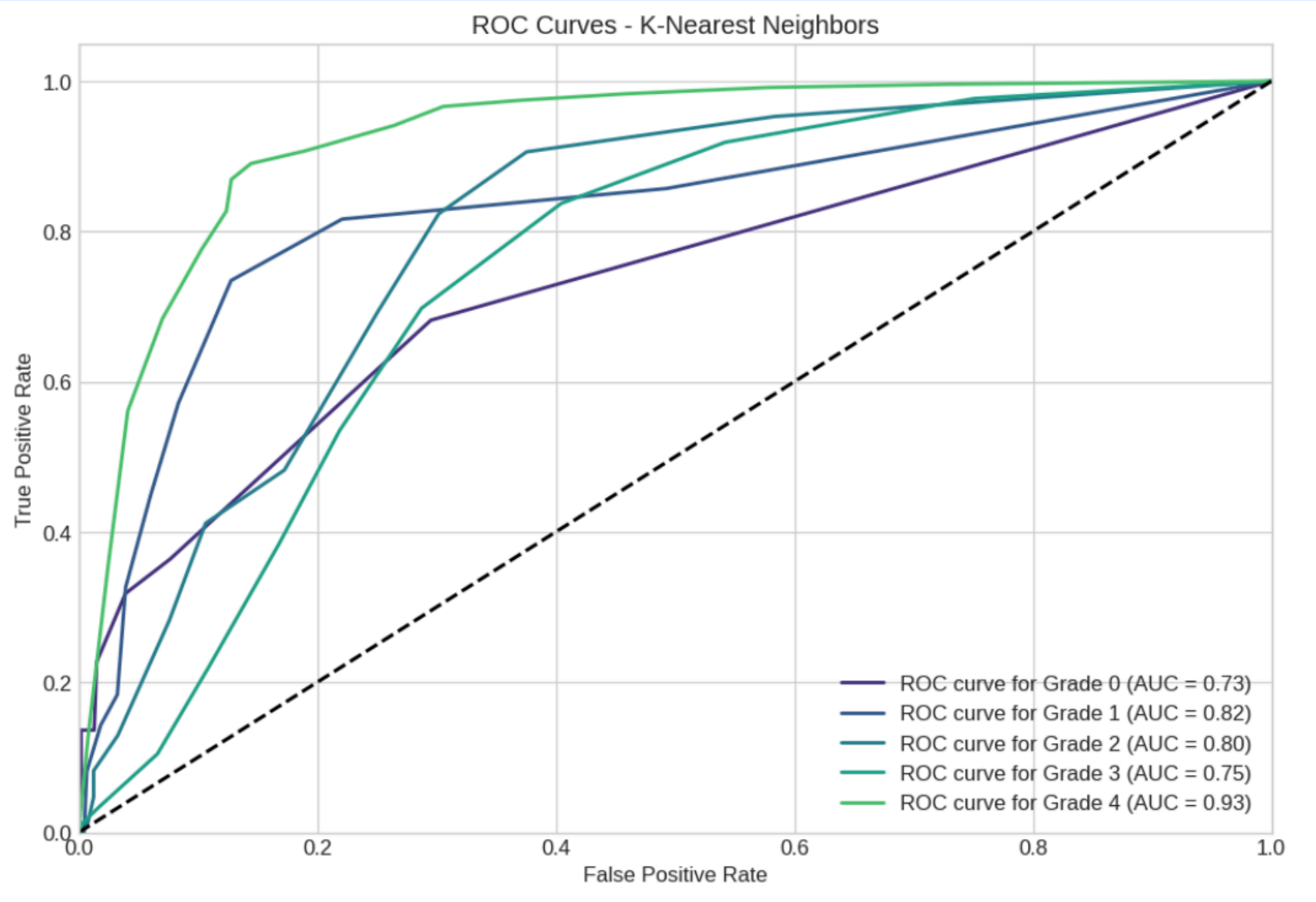
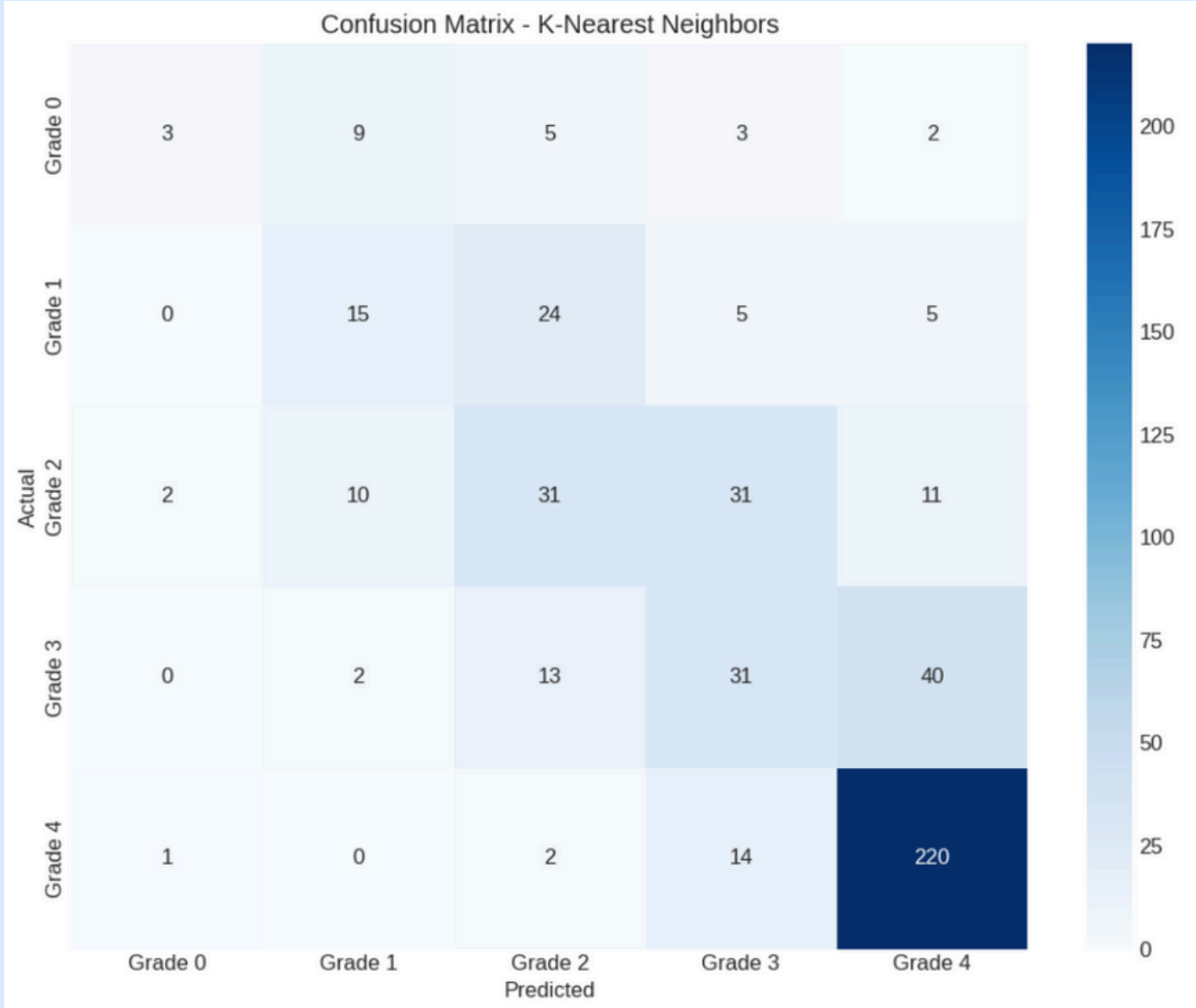


EVALUASI MODEL

K-NEAREST NEIGHBORS DENGAN HYPERPARAMETER TUNING:

----- K-Nearest Neighbors Evaluation -----

Accuracy: 0.6263
Precision (macro): 0.4981
Recall (macro): 0.4192
F1 Score (macro): 0.4348

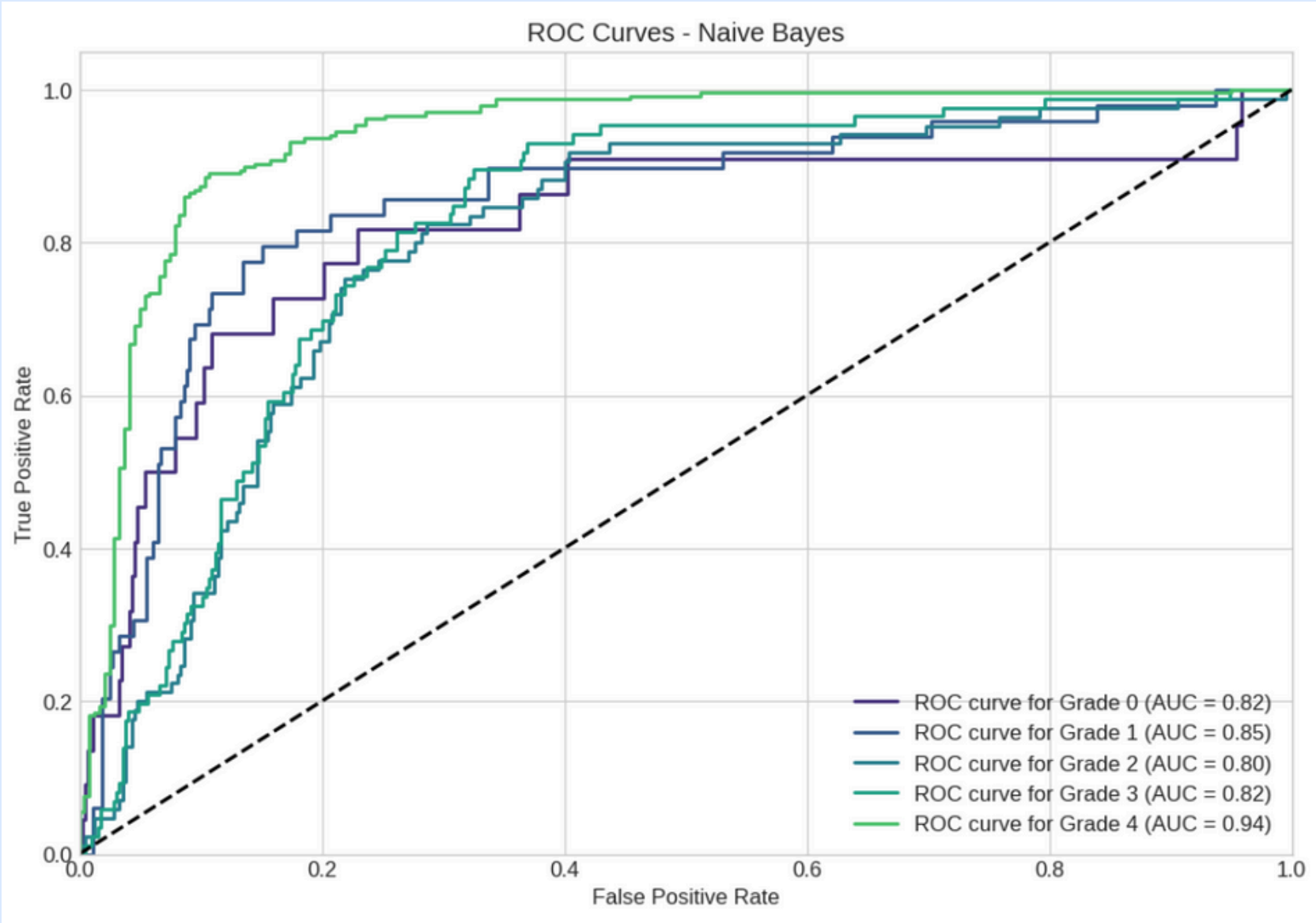
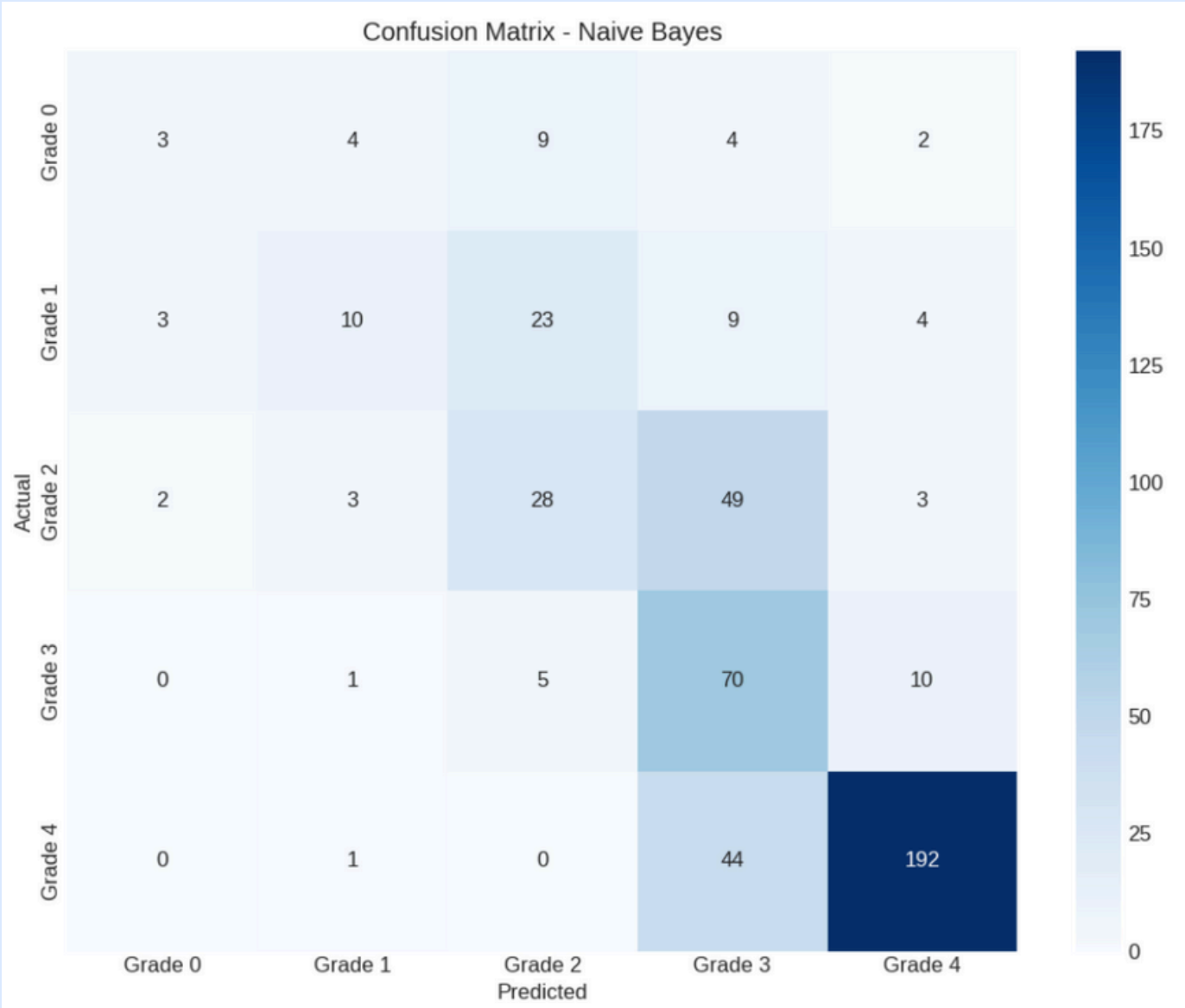


EVALUASI MODEL

NAIVE BAYES:

----- Naive Bayes Evaluation -----

Accuracy: 0.6326
Precision (macro): 0.5280
Recall (macro): 0.4588
F1 Score (macro): 0.4518

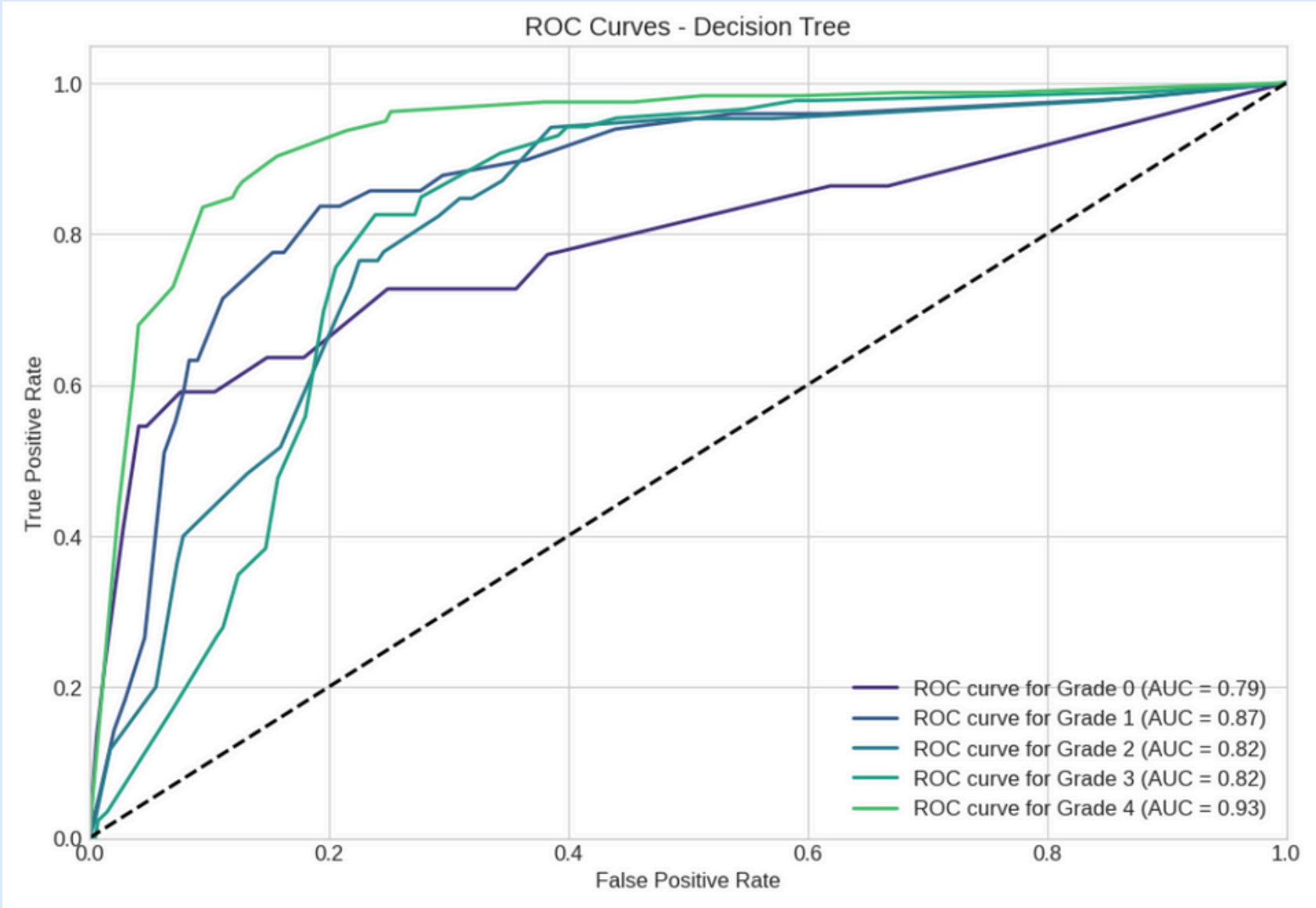
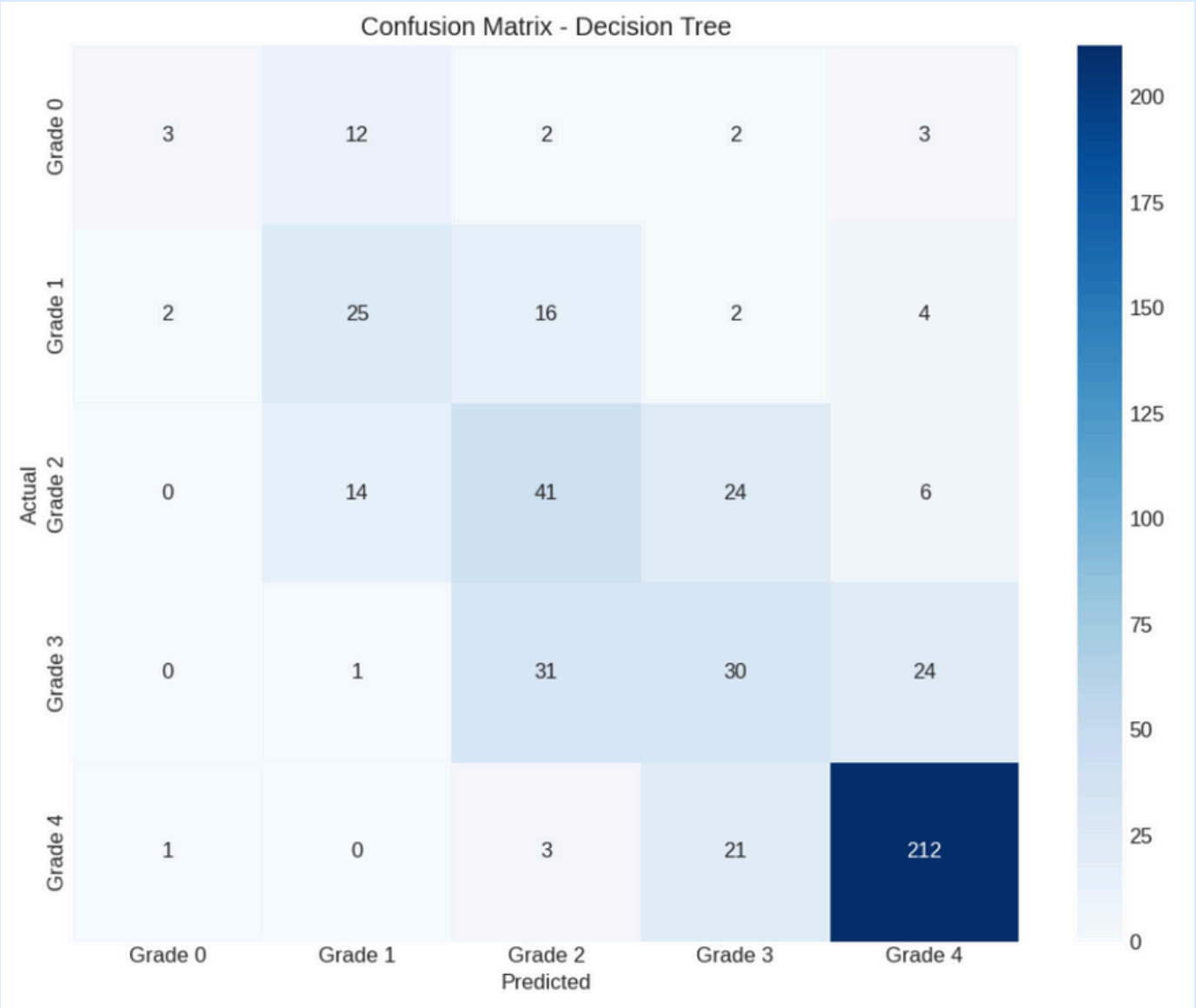


EVALUASI MODEL

DECISION TREE DENGAN HYPERPARAMETER TUNING:

----- Decision Tree Evaluation -----

Accuracy: 0.6493
Precision (macro): 0.5306
Recall (macro): 0.4745
F1 Score (macro): 0.4812



ANALISIS HASIL - PERBANDINGAN PERFORMA MODEL

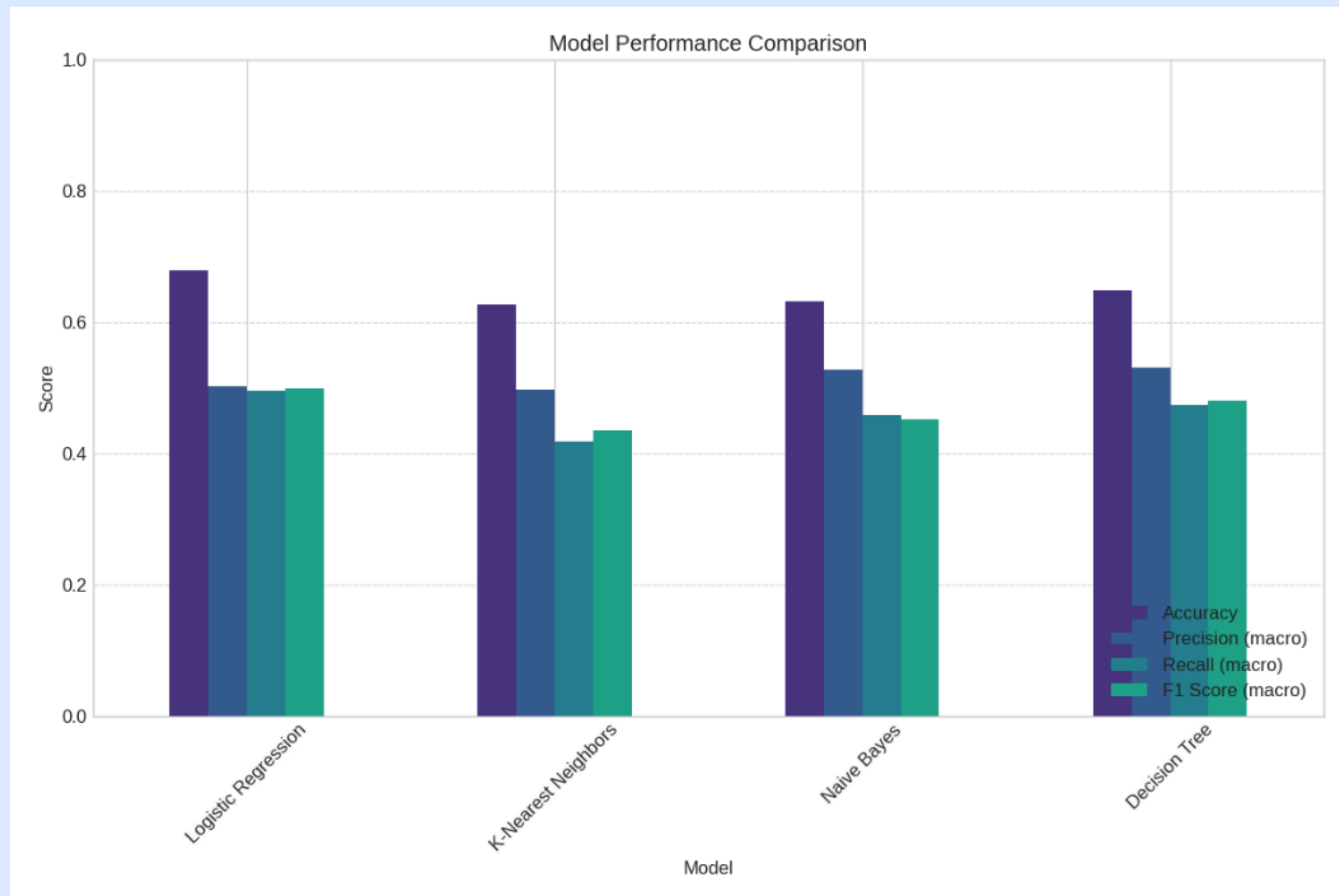


MODEL COMPARISON

Model Performance Comparison:

	Accuracy	Precision (macro)	Recall (macro)	F1 Score (macro)
Model				
Logistic Regression	0.678	0.503	0.495	0.499
K-Nearest Neighbors	0.626	0.498	0.419	0.435
Naive Bayes	0.633	0.528	0.459	0.452
Decision Tree	0.649	0.531	0.474	0.481

ANALISIS HASIL - PERBANDINGAN PERFORMA MODEL

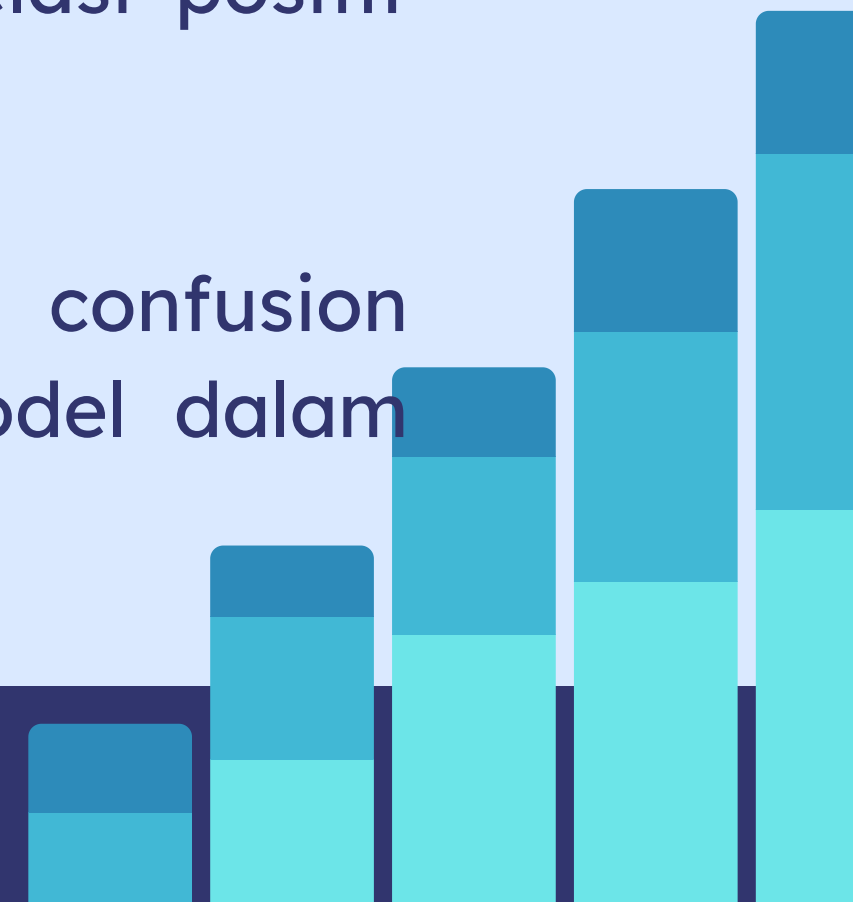


KESIMPULAN

Berdasarkan analisis klasifikasi terhadap Student Academic Performance Dataset, model Logistic Regression menunjukkan performa terbaik dibanding KNN, Naive Bayes, dan Decision Tree, dengan akurasi dan F1-score tertinggi. Ini menunjukkan bahwa hubungan antara fitur dan performa akademik siswa cenderung linear.

Fitur seperti raisedhands, VisITedResources, dan Discussion terbukti paling berpengaruh, menandakan bahwa partisipasi aktif siswa berkorelasi positif dengan hasil akademik.

Evaluasi menggunakan akurasi, precision, recall, F1-score, dan confusion matrix membantu memahami kekuatan dan kelemahan tiap model dalam mengklasifikasikan label Low, Medium, dan High.





Thank You!